

Farg'ona viloyati, O'zbekiston tumani, Qumbosti MFY
«KELAJAK TEXNOLOGIYALARI»
Nazorat ishlari — To'liq Javoblar va Baholash Mezonlari
6-sinf | 4 chorak | O'qituvchi uchun qo'llanma

Tuzuvchi:	Xo'jayev Muhammad Yusuf
Tekshiruvchi:	Maslahatchi D. Mehmonova
O'quv yili:	2026–2027
Hujjat turi:	O'qituvchi nusxasi (javoblar bilan)

Diqqat: Yashil rangdagi javoblar — to'liq namuna. O'quvchi javobi mazmunan mos kelsa, to'liq ball beriladi. So'zma-so'z mos bo'lishi shart emas.

I CHORAK NAZORAT ISHI — Oktabr oxiri

Format: Yozma test (5 savol) + Amaliy topshiriq (2 topshiriq) | Jami: 20 ball

NAZARIY QISM — 10 ball

1. Python'da try-except bloki nima uchun ishlatiladi? Misol keltiring.

try-except — dastur ishlayotganda yuz berishi mumkin bo'lgan xatolarni ushlash va dasturning to'xtab qolishini oldini olish uchun ishlatiladi.

Misol:

try:

```
son = int(input("Son kiriting: "))  
print(10 / son)
```

except ZeroDivisionError:

```
print("Nolga bo'lish mumkin emas!")
```

except ValueError:

```
print("Bu son emas!")
```

Kalit: try — xatoli bo'lishi mumkin kod; except — xato ushlanadi va alternativa bajariladi.

2. Python'da string metodlaridan split() va join() ni tushuntiring, misol keltiring.

split() — stringni belgilangan ajratuvchi bo'yicha ro'yxatga bo'ladi.

join() — ro'yxat elementlarini birlashtiradi.

Misol:

```
matn = "olma nok uzum"
```

```
ro_yxat = matn.split() # → ['olma', 'nok', 'uzum']
```

```
natiija = "-".join(ro_yxat) # → 'olma - nok - uzum'
```

Kalit: split() — qirqadi, join() — yopishtiradi.

3. Lambda funksiya nima? Oddiy funksiya bilan farqi nima?

Lambda — nomlanmagan, bitta ifodadan iborat qisqa funksiya.

Oddiy funksiya:

```
def kvadrat(x): return x**2
```

Lambda:

```
kvadrat = lambda x: x**2
```

```
print(kvadrat(5)) # → 25
```

Lambda map() va sorted() bilan birgalikda ko'p ishlatiladi.

Kalit: Lambda — qisqa va bir martalik funksiyalar uchun.

4. Python'da random modulidan foydalanib tasodifiy son olish qanday amalga oshiriladi?

```
import random
```

```
print(random.randint(1, 100)) # 1 dan 100 gacha butun son
```

```
print(random.random()) # 0 dan 1 gacha o'nlik kasr
```

```
ro_yxat = [1, 2, 3, 4, 5]
```

```
random.shuffle(ro_yxat) # ro'yxatni aralashtirib yuboradi
```

```
print(random.choice(ro_yxat)) # ro'yxatdan tasodifiy element
```

Kalit: random.randint(a, b) — amaliy sinflarda eng ko'p ishlatiladigan.

5. *args va **kwargs Python'da qanday vazifani bajaradi?

*args — funksiyaga ixtiyoriy miqdorda joylashish argumentlari uzatish imkonini beradi (tuple sifatida keladi).

**kwargs — kalit-qiyamat argumentlari uchun (dict sifatida keladi).

```
def salom(*args, **kwargs):  
    for ism in args:  
        print(f"Salom, {ism}!")  
    for k, v in kwargs.items():  
        print(f"{k}: {v}")
```

```
salom("Ali", "Vali", yosh=14, sinf="6")
```

Kalit: *args — sanab bo'lmaydigan argumentlar, **kwargs — nomlangan argumentlar.

AMALIY QISM — 10 ball

1. Python'da 10 ta tasodifiy son generatsiya qilib, ulardan eng kattasi va o'rtachasini toping.

```
import random  
sonlar = [random.randint(1, 100) for _ in range(10)]  
print("Sonlar:", sonlar)  
print("Eng katta:", max(sonlar))  
print("O'rtacha:", sum(sonlar) / len(sonlar))
```

Kalit: Baholash: random.randint va list — 2, max() — 1, sum()/len() — 2 ball

2. Foydalanuvchi kiritgan matndagi so'zlarni sanab, har bir so'z necha marta takrorlanganini chiqaruvchi dastur yozing.

```
matn = input("Matn kiriting: ").lower()  
so_zlar = matn.split()  
hisoblagich = {}  
for so_z in so_zlar:  
    hisoblagich[so_z] = hisoblagich.get(so_z, 0) + 1  
for so_z, son in hisoblagich.items():  
    print(f"{so_z}: {son} marta")
```

Kalit: Baholash: split — 1, dict mantiq — 3, chiqarish — 1 ball

II CHORAK NAZORAT ISHI — Dekabr oxiri

Format: Yozma test (5 savol) + Amaliy topshiriq (2 topshiriq) | Jami: 20 ball

NAZARIY QISM — 10 ball

1. OOP'da klass va obyekt nima? Real hayotdan misol bering.

Klass — obyekt uchun qolip (shablon). Obyekt — shu qolipdan yasalgan aniq narsa.

```
class Mashina:
    def __init__(self, marka, rang):
        self.marka = marka
        self.rang = rang
    def tavsif(self):
        print(f"{self.rang} rangli {self.marka}")
```

```
m1 = Mashina("Nexia", "oq")
```

```
m1.tavsif() # → oq rangli Nexia
```

Mashina — klass (qolip), m1 — obyekt (aniq namuna).

Kalit: *Klass = qolip, Obyekt = shu qolipdan yasalgan narsa.*

2. __init__ metodi nima uchun kerak?

__init__ — klass objekti yaratilganda avtomatik chaqiriladigan maxsus metod. U obyektning boshlang'ich qiymatlarini belgilaydi.

```
class Talaba:
    def __init__(self, ism, yosh):
        self.ism = ism
        self.yosh = yosh
```

```
t = Talaba("Zulfiya", 12)
```

```
print(t.ism) # → Zulfiya
```

Kalit: *__init__ — konstruktor, obyekt tug'ilganda ishlaydi.*

3. Vorislik (inheritance) nima? Nima uchun kerak?

Vorislik — bola klass ota klassning barcha xususiyat va metodlarini meros olishi.

```
class Hayvon:
    def ovqat(self): print("Ovqatlanmoqda")
class Mushuk(Hayvon):
    def miyov(self): print("Miyov!")
```

```
m = Mushuk()
```

```
m.ovqat() # ota klassdan meros
```

```
m.miyov() # o'zining metodi
```

Foyda: kodni qayta yozmaslik, umumiy mantig'ni markazlashtirish.

Kalit: *Vorislik = ota klass kodini bola klassga o'tkazish.*

4. __str__ maxsus metodi nima uchun ishlatiladi?

__str__ — print(obyekt) chaqirilganda ko'rsatiladigan matnni belgilaydi.

```
class Kitob:
    def __init__(self, sarlavha, muallif):
        self.sarlavha = sarlavha
        self.muallif = muallif
    def __str__(self):
        return f"{self.sarlavha} - {self.muallif}"
```

```
k = Kitob("Xamsa", "Navoiy")
print(k) # → "Xamsa" – Navoiy
```

Kalit: `__str__` — obyektning 'chiroyli' ko'rinishini belgilaydi.

5. OOP'da inkapsulyatsiya (encapsulation) nima va nima uchun muhim?

Inkapsulyatsiya — obyektning ichki ma'lumotlarini to'g'ridan-to'g'ri o'zgartirmaslik, buning o'rniga maxsus metodlar (getter/setter) orqali kirish.

```
class BankHisobi:
    def __init__(self):
        self.__balans = 0 # yashirin atribut
    def tolov(self, summa):
        if summa > 0: self.__balans += summa
    def balans(self): return self.__balans
```

Bu ma'lumot xavfsizligini ta'minlaydi.

Kalit: Inkapsulyatsiya — ichki ma'lumotni himoya qilish.

AMALIY QISM — 10 ball

1. 'Talaba' klassini yarating: ism, sinf, baholar ro'yxati atributlari bo'lsin. O'rtacha bahoni hisoblovchi metod qo'shing.

```
class Talaba:
    def __init__(self, ism, sinf):
        self.ism = ism
        self.sinf = sinf
        self.baholar = []
    def baho_qosh(self, baho):
        self.baholar.append(baho)
    def ortacha(self):
        if self.baholar:
            return sum(self.baholar) / len(self.baholar)
        return 0
    def __str__(self):
        return f"{self.ism} ({self.sinf}-sinf) – o'rtacha: {self.ortacha():.1f}"
```

```
t = Talaba("Sardor", 6)
t.baho_qosh(90)
t.baho_qosh(85)
print(t)
```

Kalit: Baholash: klass va `__init__` — 2, metod — 2, `__str__` — 1 ball

2. Hayvon (Animal) ota klassidan Mushuk va It bola klasslarini yarating, har birida o'ziga xos tovush metodi bo'lsin.

```
class Hayvon:
    def __init__(self, ism): self.ism = ism
    def ovqat(self): print(f"{self.ism} ovqatlanmoqda")
```

```
class Mushuk(Hayvon):
    def tovush(self): print("Miyov!")
```

```
class It(Hayvon):
```

```
def tovush(self): print("Vov-vov!")
```

```
m = Mushuk("Murzik")
```

```
i = It("Barqut")
```

```
m.ovqat(); m.tovush()
```

```
i.ovqat(); i.tovush()
```

Kalit: Baholash: ota klass — 2, vorislik to'g'ri — 2, polimorfizm — 1 ball

III CHORAK NAZORAT ISHI — Mart oxiri

Format: Yozma test (5 savol) + Amaliy topshiriq | Jami: 20 ball

NAZARIY QISM — 10 ball

1. Pygame'da o'yin siklini (game loop) tushuntiring.

O'yin sikli — o'yin davomida cheksiz ishlaydi va uch vazifani bajaradi:

- Voqealarni tekshirish (events) — klaviatura/sichqoncha bosilishi
- Ma'lumotlarni yangilash — o'yinchi/dushman pozitsiyasini o'zgartirish
- Ekranni qayta chizish — yangilangan kadrni ko'rsatish

while ishlamoqda:

events() → update() → draw()

Bu 60 FPS (kadr/sekund) tezlikda ishlaydi.

Kalit: O'yin sikli = tekshir → yangilash → chiz — cheksiz.

2. Pygame'da to'qnashuv aniqlash (collision detection) qanday amalga oshiriladi?

Pygame'da collide_rect() metodi yoki spritecollide() funksiyasi ishlatiladi.

Misol:

oyinchi_rect = pygame.Rect(100, 100, 50, 50)

dushman_rect = pygame.Rect(120, 110, 50, 50)

```
if oyinchi_rect.collide_rect(dushman_rect):  
    print("To'qnashdi!")
```

Har bir kadrda tekshiriladi.

Kalit: collide_rect() — ikki to'rtburchak kesishishini tekshiradi.

3. Python Turtle'da ekranga 5 xil rangli pentagon (5 burchakli) chizish uchun qanday kod yoziladi?

```
import turtle
```

```
t = turtle.Turtle()
```

```
ranglar = ['red', 'blue', 'green', 'orange', 'purple']
```

```
for rang in ranglar:
```

```
    t.color(rang)
```

```
        for _ in range(5):
```

```
            t.forward(80)
```

```
            t.right(72) # 360/5 = 72
```

```
        t.penup(); t.forward(100); t.pendown()
```

```
turtle.done()
```

Kalit: Pentagon: 5 ta tomon, har biri 72 darajaga burish.

4. Pygame'da fon rangini o'rnatish va ekranni yangilash buyruqlarini yozing.

```
import pygame
```

```
pygame.init()
```

```
ekran = pygame.display.set_mode((800, 600))
```

```
# Fon rangi:
```

```
ekran.fill((30, 30, 30)) # qoramtir fon (RGB)
```

```
# Ekranni yangilash:
```

```
pygame.display.flip() # yoki pygame.display.update()
```

Kalit: `fill()` — rangi RGB formatda; `flip()` — ekranni yangilaydi.

5. O'yin dasturlashda FPS (Frames Per Second) nima va uni qanday nazorat qilish mumkin?

FPS — sekundiga ko'rsatiladigan kadrlar soni. Yuqori FPS = silliq animatsiya.

Pygame'da:

```
clock = pygame.time.Clock()
```

```
# o'yin siklida:
```

```
clock.tick(60) # FPS ni 60 ga cheklaydi
```

60 FPS oddiy o'yinlar uchun, 144 FPS professional darajada.

Kalit: `clock.tick(60)` — o'yin juda tez ishlamasin deb cheklaydi.

AMALIY QISM — 10 ball

1. Pygame'da oq fonda qizil to'p chapdan o'ngga harakatlanadigan oddiy animatsiya yarating.

```
import pygame
pygame.init()
ekran = pygame.display.set_mode((800, 400))
clock = pygame.time.Clock()
x, y = 50, 200
ishlamoda = True
while ishlamoqda:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            ishlamoqda = False
    ekran.fill((255, 255, 255))
    pygame.draw.circle(ekran, (255, 0, 0), (x, y), 30)
    x += 3
    if x > 800: x = 0
    pygame.display.flip()
    clock.tick(60)
pygame.quit()
```

Kalit: Baholash: oyna — 1, sikl — 2, harakat mantig'i — 2 ball

2. Turtle bilan 6 ta rangli uchburchak chizib, ularni aylana bo'ylab joylashtiring.

```
import turtle
t = turtle.Turtle(); t.speed(0)
ranglar = ['red', 'blue', 'green', 'orange', 'purple', 'pink']
for i, rang in enumerate(ranglar):
    t.color(rang)
    for _ in range(3):
        t.forward(60); t.left(120)
    t.penup(); t.right(60); t.forward(120); t.pendown()
turtle.done()
```

Kalit: Baholash: uchburchak to'g'ri — 3, rang — 1, joylashuv — 1 ball

IV CHORAK — YAKUNIY NAZORAT — May oxiri

Format: Yakuniy loyiha taqdimoti + Yillik test | Jami: 30 ball

NAZARIY QISM — 10 ball

1. JSON formatida ma'lumot nima ko'rinishda saqlanadi? Python'da o'qish va yozish qanday amalga oshiriladi?

JSON — matn ko'rinishidagi kalit-qiyamat formatidagi ma'lumot.

Misol JSON: {"ism": "Ali", "yosh": 14, "baholar": [90, 85, 95]}

```
import json
# Yozish:
with open("data.json", "w") as f:
    json.dump({"ism": "Ali"}, f)
# O'qish:
with open("data.json") as f:
    data = json.load(f)
print(data["ism"])
```

Kalit: `json.dump()` — yozish, `json.load()` — o'qish.

2. API nima? requests kutubxonasi bilan qanday ma'lumot olinadi?

API — dasturlar orasida ma'lumot almashish interfeysi. requests kutubxonasi HTTP so'rovlar yuborish uchun:

```
import requests
javob = requests.get("https://api.example.com/data")
print(javob.status_code) # 200 = muvaffaqiyatli
data = javob.json() # JSON'ga o'girish
print(data)
```

Kalit: `requests.get(url).json()` — API'dan ma'lumot olishning eng qisqa yo'li.

3. Yil davomida eng ko'p foydalangan Python tushunchasi qaysi bo'ldi va nima uchun?

Shaxsiy javob: "Men [tushuncha]ni eng ko'p ishlatdim, chunki... [sabab]. Bu tushuncha menga [loyiha/vazifada] yordam berdi."

Kalit: To'g'ri/ noto'g'ri javob yo'q — asoslash muhim.

4. OOP'ning qanday afzalliklari bor? Nima uchun katta loyihalarda ishlatiladi?

OOP afzalliklari: kodni qayta ishlatish (inheritance), ma'lumotni himoya qilish (encapsulation), real dunyo obyektlarini modellash, buyuk loyihalarda tartib saqlash.

Misol: minglab fayldan iborat dasturlarda OOP bez kodni tartibli va tushunarli saqlaydi.

Kalit: `OOP = tartib, xavfsizlik, qayta ishlatish.`

YAKUNIY LOYIHA — 20 ball (baholash rubrikasi)

- 1. Loyiha mavzusi aniq va foydali (5 ball) — OOP, API yoki grafika ishlatilganmi? Kalit: kamida bitta o'rganilgan texnologiya amalda qo'llanilsin.
- 2. Texnik bajarilishi: kod ishlayaptimi? OOP/API/Pygame to'g'ri ishlatilganmi? (8 ball) — Eng yuqori ball ishlaydigan loyihaga.

- 3. Taqdimot sifati: kod tushuntirilsinmi, natija ko'rsatilsinmi? (5 ball)
- 4. Jamoaviy ish: agar guruhda ishlangan bo'lsa, vazifalar taqsimlangan bo'lsin (2 ball)

Ushbu hujjat «Kelajak texnologiyalari» to'garagi uchun tayyorlangan rasmiy o'qituvchi qo'llanmasi. Xo'jayev Muhammad Yusuf, Farg'ona viloyati, O'zbekiston tumani, Qumbosti MFY, 45-sonli maktab, 6-sinf. 2026-yil.